

CSE235 Week 13: Installing External Packages and NumPy

One of the main reasons Python is and stays popular is the wealth of third-party libraries (often referred to as packages in Python) available for it. Today, we'll discuss how to install packages and create virtual environments for Python. We'll then discuss two of the most popular packages, NumPy and Matplotlib.

Pip

Pip is a recursive acronym that means "Pip installs packages". It has been the defacto standard for installing third party software packages in Python since it was released in 2008. Pip interacts with PyPI (Python Package Index), a remote repository of Python packages, as its primary source for installing packages. Pip can be connected to other repositories as well, provided they provide an interface that pip can interact with. We will only be using packages available on PyPI in this course.

Pip is a command line tool. You can run it in one of the following ways:

Windows:

```
pip install package_name  
# or  
python -m pip install package_name
```

Mac/Linux

```
pip3 install package_name  
# or  
python3 -m pip install package_name
```

The `python -m` syntax tells Python to look for and run a preinstalled Python module. Pip is a preinstalled with all recent versions of Python 3. The `install` command tells it to look for the specified package in the PyPI repo. If it finds that package, it downloads and installs the package. By default, packages are installed into the global Python environment.

We will discuss virtual environments in a coming section. Let's install our first package, NumPy. The command to do so is:

Windows:

```
pip install numpy
```

Mac/Linux:

```
pip3 install numpy
```

This will install NumPy onto your system. We'll discuss using NumPy in a couple of sections. Pip is also used to uninstall packages. If you decide to unistall NumPy, the command would be:

Windows:

```
pip uninstall numpy
```

Mac/Linux:

```
pip3 uninstall numpy
```

When installing and removing packages from the global environment, these commands can be run anywhere, and they affect the whole system.

Some Python projects may require many external packages, along with specific versions of these packages to work. A `requirements.txt` file is a list of packages that are required for a Python project to work. Pip has the ability to read a `requirements.txt` file and install all of the packages listed within it. To do so, run the following command:

Windows:

```
pip -r requirements.txt
```

Mac/Linux:

```
pip3 -r requirements.txt
```

However, this may install a large number of packages that you only need for a single project into your global environment. Python is a constantly changing language, and version updates may break existing packages. Therefore, it is often recommended that packages are not installed into the global environment. Virtual environments were created to address this issue. In the next section, we'll discuss how to create and use virtual

environments.

Virtual Environments

A virtual environment is an isolated installation of Python. Typically, a folder called `.venv` is used to store the files for the virtual environment. The environment contains its own version of the Python interpreter and pip. When using the version of Python and pip in the virtual environment, external packages installed are installed to the `.venv` folder instead of the entire system. This allows you to isolate different projects and keep the global environment clean. The `venv` module is used to create virtual environments. Typically, the `.venv` folder is created within the folder containing the project that the virtual environment is for. Assuming your terminal is in the folder of your project, this command is used to create a virtual environment:

Windows:

```
python -m venv .venv
```

Mac/Linux:

```
python3 -m venv .venv
```

This creates the virtual environment files, but does not automatically switch your terminal to use that environment. The following commands are used to activate the virtual environment, telling your terminal to use that environment instead of the global one:

Windows (Command Prompt):

```
.venv\Scripts\activate.bat
```

Windows (PowerShell):

```
.venv\Scripts\Activate.ps1
```

Mac/Linux:

```
source .venv/bin/activate
```

You will know your terminal is using a virtual environment due to `(.venv)` appearing before your terminal prompt. To stop using the virtual environment, run `deactivate`.

Most IDEs and even rich text editors like VS Code have the capability to create and activate a virtual environment for you, as well. The process

to do so varies by IDE.

NumPy

NumPy is a mathematics library for Python. One of the biggest problems with Python is that, being an interpreted language, it is very slow. Additionally, lists, while convenient to use, slow down program execution because they can hold different data types. Even if you create a list of all integers, Python does not know that is your intention, and will always have to treat every index in the list as though it could be anything. NumPy is a C library with bindings that make it usable in Python. Compiled C code is much faster than Python, and there are many performance optimizations that can be done in C that are simply impossible in Python.

NumPy Arrays

One of the main reasons to use NumPy is it's `Array` data type. Python lists are like arrays, but they can store elements of any data type. NumPy arrays, on the other hand, function more like arrays in other languages, where only a single data type must be stored. This reduces the burden of determining the data type for every element in the array.

Because NumPy arrays enforce a single data type, the computer can store the data contiguously in memory, making mathematical operations incredibly fast. To use NumPy in your code, you must first import it. The standard convention is to import it under the alias `np`:

```
import numpy as np
```

Creating Arrays

You can create a NumPy array by passing a standard Python list into the `np.array()` function:

```
# creating a 1D array
my_list = [1, 2, 3, 4, 5]
my_array = np.array(my_list)
print(my_array)

# creating a 2D array (a matrix)
matrix_list = [[1, 2, 3], [4, 5, 6]]
my_matrix = np.array(matrix_list)
print(my_matrix)

# output:
# [[1 2 3]]
```

```
# [4 5 6]]
```

Element-wise Operations (Vectorization)

In standard Python, if you want to multiply every number in a list by 2, you have to use a `for` loop or a list comprehension. NumPy allows you to perform operations on the entire array at once, which is called vectorization.

```
# standard Python list (requires a loop)
python_list = [1, 2, 3]
doubled_list = [x * 2 for x in python_list]

# NumPy array (direct math)
data = np.array([1, 2, 3])
doubled_data = data * 2
# prints: [2 4 6]
print(doubled_data)

# you can also add arrays together
array1 = np.array([1, 2, 3])
array2 = np.array([4, 5, 6])
# output: [5 7 9]
print(array1 + array2)
```

Matplotlib

Matplotlib is the go-to library for data visualization in Python. It allows you to transform your data and NumPy arrays into highly customizable charts, graphs, and plots.

To use it, we typically import the `pyplot` submodule using the alias `plt`. If you want to use it alongside your data, your import statements will usually look like this:

```
import numpy as np
import matplotlib.pyplot as plt
```

Creating a Basic Line Plot

The most common way to use Matplotlib is to plot *x* and *y* coordinates. The `plt.plot()` function connects these coordinates with a line, and `plt.show()` displays the window containing the graph.

```
# sample data
x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 6, 8, 10])

# create a line plot
plt.plot(x, y)

# it is best practice to always label your graphs
plt.title("My First Line Plot")
plt.xlabel("X Axis Label")
plt.ylabel("Y Axis Label")

# Display the plot
plt.show()
```

Creating a Scatter Plot

If you want to display individual data points without connecting them with a line, you can use `plt.scatter()` instead:

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([5, 2, 9, 1, 7])

# Create a scatter plot
plt.scatter(x, y, color='red', marker='o')
plt.title("Simple Scatter Plot")
plt.xlabel("Independent Variable")
plt.ylabel("Dependent Variable")

plt.show()
```

Conclusion

By using `pip`, we can easily install third-party code packages from the Python Package Index (PyPI). To keep our computer systems organized and prevent different projects from conflicting with one another, we learned how to isolate our work using Virtual Environments (`.venv`). The abundance of third-party software is one of the major reasons why Python is so widely used. Afterward, we looked at two of the most commonly used Python packages, NumPy and Matplotlib. NumPy provides high-performance, fixed-type numerical arrays that allow us to perform fast, vector-based math. Matplotlib is used to create charts and graphs.